Facultad de Defensa Nacional

Licenciatura en CIBERDEFENSA

Lenguajes de Programación

El lenguaje C

Manual de Referencia v 1.2

Docentes:

Ing. Sebastián Balestrini

Esp. Ing. Gustavo Di Stefano

Carga horaria: 64 hs

Introducción a C **3**
El primer programa en C: Hola Mundo **4**
Variables y Tipos en C **5**
Declaración y uso de variables 6
Tipos de datos en C **7**
Enteros sin signo (unsigned) 7
Desbordamiento (Overflow) 7
Números de punto flotante 8
Constantes **9**
Comentarios en C **10**
Comentarios de una sola línea. 10
Comentarios de varias líneas 11
Buenas prácticas 11
Operadores en C **11**
Operadores aritméticos 12
Binarios (requieren dos operandos) 12
Unarios (un solo operando) 12
Operadores de comparación 13
Operadores lógicos 13
Operadores de asignación compuestos 13
Precedencia de operadores 14
Condicionales en C **15**
if 15
switch 16
Bucles en C **17**
Ciclo for 17
Ciclo while 17
Ciclo do while 18
Saliendo de un bucle con break 18
Punteros **19**
Operadores asociados a punteros 19
1. Operador de referencia (&): 19
2. Operador de desreferencia (*): 19
Arreglos **20**
Arreglos unidimensionales (Vectores) 21
Arreglos bidimensionales (Matrices) 21
Arreglos multidimensionales 21
Recorrido de un vector 22
Recorrido de una matriz 22

Búsqueda lineal en un vector 23
Ordenamiento (Bubble Sort) 23
Funciones y Modularización en C **25**
Funciones 25
Procedimientos 25
Tipo de dato void 25
Variables locales y globales 26
Pasaje de parámetros 26
Funciones recursivas 27

Manual de Referencia del Lenguaje C

Introducción a C

El lenguaje C fue creado en 1972 por Dennis Ritchie en los Laboratorios Bell (Bell Labs), como evolución de los lenguajes B y BCPL. Su objetivo inicial fue proporcionar un lenguaje flexible y eficiente para el desarrollo del sistema operativo UNIX, el cual hasta ese momento estaba escrito en ensamblador.

Gracias a C, UNIX pudo ser reescrito casi en su totalidad en un lenguaje de alto nivel, lo que facilitó su portabilidad hacia diferentes arquitecturas de hardware, un logro revolucionario en esa época.

Con el tiempo, C se consolidó como un lenguaje de propósito general, ampliamente adoptado en ámbitos académicos e industriales. Su eficiencia, control sobre la memoria y cercanía al hardware lo convierten en la base de muchos otros lenguajes posteriores como C++, C#, Java, Objective-C y hasta lenguajes modernos como Go o Rust, que heredan ideas de su diseño.

Además, C es un **lenguaje compilado**, lo que significa que el código fuente escrito por el programador debe pasar por un proceso de traducción mediante un **compilador**. Este convierte el código C en instrucciones de bajo nivel (código objeto y, finalmente, un ejecutable) que la computadora entiende y ejecuta directamente. Esto permite que los programas en C sean altamente eficientes en tiempo de ejecución, a diferencia de otros lenguajes interpretados que se ejecutan línea por línea.

Hoy en día, C sigue siendo una herramienta fundamental en áreas donde la precisión y el rendimiento son críticos, como:

- **Sistemas embebidos** (microcontroladores, IoT, dispositivos médicos).
- **Sistemas operativos** (Linux, Windows, macOS tienen grandes núcleos escritos en C).
- **Drivers y controladores de hardware**.

- **Aplicaciones críticas** en telecomunicaciones, automotriz, aeroespacial y médica.

En definitiva, C no solo fue clave en el nacimiento de UNIX, sino que estableció las bases de la informática moderna, influyendo en casi todos los lenguajes y sistemas que usamos actualmente.

El primer programa en C: *Hola Mundo*

Uno de los ejemplos más clásicos al aprender cualquier lenguaje de programación es el programa **"Hola Mundo"**. En C, este programa luce así:

hola.c

```c
#include <stdio.h>

int main(void) {
    printf("Hola Mundo!");
    return 0;
}
```

Descripción del código

1. **Inclusión de la librería estándar (`#include <stdio.h>`)**
   - `stdio` significa *standard input/output* (entrada/salida estándar).
   - Esta librería nos da acceso a funciones de entrada y salida, como `printf()`.
   - C tiene un núcleo muy pequeño, y las funcionalidades adicionales se proveen mediante **librerías**. Algunas son creadas por otros programadores, mientras que otras vienen integradas en el compilador.
2. **La función `main()`**
   - Todo programa en C comienza a ejecutarse en la función `main()`.
   - Su forma estándar es:

```
int main(void) {
    // instrucciones
}
```

3. **La función `printf()`**
    - Esta función pertenece a la librería stdio.h.
    - Se usa para imprimir texto en pantalla.
    - Su definición en la librería es algo así:

      `int printf(const char *format, ...);`
    - No es necesario entender toda esta definición aún, pero en resumen:
        → `const char *format` significa que espera recibir una cadena de texto.
        → Los `...` indican que puede recibir más argumentos opcionales (ejemplo: variables, números, etc.).

4. **La sentencia `return 0;`**
    - Indica que el programa terminó exitosamente.
    - Si devolvemos otro número, podría significar un error o condición especial.

En resumen

- `#include <stdio.h>` importa la librería necesaria para usar printf().
- int main(void) define el punto de inicio del programa.
- printf("¡Hola Mundo!"); imprime un mensaje en pantalla.
- return 0; informa al sistema operativo que el programa finalizó correctamente.

Cuando ejecutamos este programa, la salida será:

```
¡Hola Mundo!
```

Variables y Tipos en C

En C, toda **variable** debe estar asociada a un **tipo de dato** definido en el momento de su declaración.

Esto se debe a que **C es un lenguaje de tipado estático y compilado**, lo que significa que:

- El compilador necesita saber de antemano qué tipo de valores almacenará la variable.
- Una vez declarado, el tipo de una variable no puede cambiar.

Esto lo diferencia de lenguajes interpretados como Python, JavaScript o PHP, donde las variables pueden cambiar de tipo dinámicamente.

Declaración y uso de variables

En el siguiente ejemplo podemos ver dos variables de tipo entero, una sin valor inicial y la siguiente con un valor inicial.

```
int edad;
int edad_adulto = 18;
```

Reglas para los nombres de variables:

- Pueden contener letras (mayúsculas o minúsculas), dígitos y guiones bajos _.
- No pueden comenzar con un número.
- Ejemplos válidos: EDAD, Edad10, _contador.
- Ejemplo inválido: 1edad.

Ejemplo: asignación y advertencia del compilador

```
#include <stdio.h>

int main(void) {
    int edad = 0;
    edad = 37.2;   // El compilador advertirá la conversión
    printf("%d", edad);
    return 0;
}
```

En este caso, el compilador mostrará una advertencia, ya que 37.2 es un número decimal (float) y se está intentando asignar a una variable entera (int). El valor se convierte automáticamente en 37.

Tipos de datos en C

C provee varios **tipos de datos primitivos** para enteros y números con punto flotante.

Tipo	Tamaño (bytes)	Rango sin signo
char	1	-128 a 127
int	2	-32,768 a 32,767
long	4	-2147483648 a 2147483648
float	4	3.4E-38 a 3.4E+38
double	8	1.7E-308 a 1.7E+308

Notas:

- **char** suele usarse para caracteres ASCII, pero también almacena números pequeños.

- El tamaño exacto depende del compilador y la arquitectura. En sistemas i686 (x86 de 32 bits) y x86_64 (64 bits) **int** ocupa 4 bytes (32 bits) mientras que Arduino (basado en AVR, como el ATmega328P de Arduino Uno) **int** solo ocupa 2 bytes .

Enteros sin signo (unsigned)

Al anteponer **unsigned**, los valores comienzan desde 0 y duplican el rango positivo

Tipo	Rango
unsigned char	0 a 255
unsigned int	0 a 65535
unsigned long	0 a 4294967295

Desbordamiento (Overflow)

En C, el compilador no protege contra desbordamientos. Ejemplo con unsigned char:

```c
#include <stdio.h>

int main(void) {
    unsigned char j = 255;
    j = j + 10;
    printf("%u", j); // Resultado: 9
    return 0;
}
```

En el ejemplo, la variable j está declarada como **unsigned char**, lo que le da un rango de 0 a 255. Al sumarle 10, se supera el límite superior (255), lo que provoca un overflow. Dado que unsigned char usa 8 bits, el resultado se ajusta automáticamente mediante aritmética modular (wraparound), calculándose como 265 % 256 = 9.

Advertencias cuando se declara el tipo equivocado

Cuando se declara una variable y se inicializa con el valor equivocado, el compilador gcc debería advertir lo siguiente:

```c
#include <stdio.h>

int main(void) {
    char j = 1000;
    return 0;
}
```

warning: overflow in conversion from 'int' to 'char' changes value from '1000' to '-24' [-Woverflow]

4 | char j = 1000;

Números de punto flotante

Para representar números decimales o muy grandes/pequeños:

Tipo	Rango
float	6-7 dígitos
double	15-16 dígitos
long double	18+ dígitos

Ejemplo

```c
#include <stdio.h>

int main(void) {
    float a = 1.29e-3;   // 0.00129
    double b = -2.3e+5;  // -230000
    printf("%f %f", a, b);
    return 0;
}
```

El código declara dos variables: a como float con valor 1.29e-3 (equivalente a 0.00129) y b como double con valor -2.3e+5 (equivalente a -230000). Al imprimirlas con printf("%f %f", a, b), los valores se muestran en notación decimal: %f interpreta ambos números correctamente, mostrando 0.001290 para a y -230000.000000 para b.

Constantes

En C existen dos formas principales de declarar constantes:

1. Usando la palabra clave **const**
    - Se declara una variable cuyo valor no puede modificarse después de la inicialización.
    - El compilador garantiza que el valor se mantenga constante.

```c
#include <stdio.h>

int main(void) {
    const int MAX = 100;
    // instrucciones
    return 0;
}
```

Aquí MAX será siempre 100 durante toda la ejecución del programa.

2. Usando la directiva del preprocesador **#define**
    - Se utiliza para definir macros o valores constantes antes de la compilación.

- El preprocesador reemplaza todas las apariciones de la constante por su valor literal.

```
#include <stdio.h>

#define MAX 100

int main(void) {
     // instrucciones
    return 0;
}
```

En este caso, antes de compilar, cada vez que aparezca MAX, será sustituido por 100.

Diferencias clave entre const y #define:

- const respeta los tipos de datos y es gestionado por el compilador.
- #define no tiene tipo, ya que solo realiza una sustitución textual.

Comentarios en C

En C existen dos formas principales de escribir comentarios en el código. Los comentarios no son ejecutados por el compilador: sirven para explicar, documentar o desactivar temporalmente partes del programa.

Comentarios de una sola línea.

Se escriben con //. Todo lo que sigue en esa línea será ignorado por el compilador.

```
#include <stdio.h>

// Este programa imprime un mensaje en pantalla

int main() {
    printf("Hola Mundo\n"); // también puedo comentar aquí
    return 0;
}
```

Uso típico:

- Explicar brevemente qué hace una instrucción.
- Anotar ideas rápidas.

Comentarios de varias líneas

Se escriben entre /* y */. Pueden abarcar una o más líneas.

```
/*
Este programa imprime un mensaje en pantalla.
Ejemplo clásico de introducción a C.
Autor: Gus Di Stefano                */

#include <stdio.h>

int main() {
    printf("Hola Mundo\n");
    return 0;
}
```

Uso típico:

- Describir bloques de código.
- Documentar funciones o programas enteros.
- Deshabilitar temporalmente varias líneas de código.

Buenas prácticas

- Usar // para comentarios cortos y simples.
- Usar /* ... */ para descripciones largas o documentación.
- Evitar comentar obviedades (ejemplo: i = i + 1; // suma 1 a i).
- Los comentarios deben ayudar a entender el por qué del código, no solo el qué.

Operadores en C

C ofrece una amplia variedad de operadores, que son símbolos especiales usados para realizar operaciones con datos.

Se pueden clasificar en varios grupos:

- Operadores aritméticos
- Operadores de comparación

- Operadores lógicos
- Operadores de asignación compuestos
- Operadores bitwise (a nivel de bit)
- Operadores de punteros
- Operadores de estructura
- Operadores misceláneos

En esta sección veremos los más usados.

Operadores aritméticos

Binarios (requieren dos operandos)

Operador	Nombre	Ejemplo
=	asignación	a = 1
+	suma	a + b
-	resta	a - b
*	multiplicación	a * b
/	división	a / b
%	módulo	a % b

Unarios (un solo operando)

Operador	Nombre	Ejemplo
++	incremento	a++ o ++a
--	decremento	a-- o --a

Diferencia entre a++ y ++a:

```
int a = 2;
int b;

b = a++; // b es 2, a es 3
b = ++a; // b es 4, a es 4
```

Es decir:

- a++ → incrementa después de usar el valor.
- ++a → incrementa antes de usar el valor.

Operadores de comparación

Operador	Nombre	Ejemplo
==	igual a	a == 1
!=	diferente de	a != b
>	mayor que	a > b
<	menor que	a < b
>=	mayor o igual que	a >= b
<=	menor o igual que	a <= b

Operadores lógicos

Operador	Nombre	Ejemplo
!	NOT	!a
&&	AND	a && b
||	OR	a || b

Operadores de asignación compuestos

Operador	Ejemplo	Equivale a
+=	a += b	a = a + b
-=	a -= b	a = a - b
*=	a *= b	a = a * b

Operador	Ejemplo	Equivale a
+=	a += b	a = a + b
/=	a /= b	a = a * b
%=	a %= b	a = a % b

Precedencia de operadores

Con todos los operadores disponibles en C (aritméticos, lógicos, relacionales, bitwise, de punteros, etc.), debemos prestar atención al **orden en que se ejecutan** cuando aparecen en una misma expresión.

```
int a = 2;
int b = 4;
int c = b + a * a / b - a;
```

¿Cuál es el valor de c?

¿Se ejecuta la suma antes que la multiplicación y la división?

En C existe un conjunto de reglas conocidas como **precedencia de operadores**, que define qué operaciones se ejecutan primero. Además, cada operador tiene una **asociatividad**, que indica si se evalúan de izquierda a derecha o de derecha a izquierda.

En orden de **menor a mayor precedencia**, los operadores básicos son:

1. **Asignación** `=`
2. **Suma y resta binaria** `+ -`
3. **Multiplicación y división** `* / %`
4. **Unarios** `+ - ++ --`

La **asociatividad** es generalmente de **izquierda a derecha**, excepto en los **unarios** y en la **asignación**, que son de derecha a izquierda.

El ejemplo `c = b + a * a / b - a` se evalúa así:

1. `a * a = 4`
2. `4 / b = 1`
3. `b + 1 - a = 4 + 1 - 2`
4. Resultado final: `c = 3`

Condicionales en C

En cualquier lenguaje de programación necesitamos **tomar decisiones** en base a condiciones.

En C contamos principalmente con:

a. **`if`** / **`else`**
b. **`switch`**

if

La sentencia **`if`** permite ejecutar un bloque de código si una condición se cumple:

```
int a = 1;
if (a == 1) {
  /* hacer algo */
}
```

Podemos agregar un bloque **`else`** para cubrir el caso contrario:

```
int a = 1;
if (a == 2) {
   /* hacer algo */
} else {
   /* hacer esto otro */
}
```

Atención a un error común: No confundir el **operador de asignación** **=** con el **operador de comparación** **==**.

```
int a = 0;
if (a = 0) {
  /* nunca será invocado */
}
```

¿Por qué pasa esto?

Porque a = 0, asigna el valor 0 a a. El resultado de la asignación es 0 → lo cual se evalúa como **falso**.

En C, `0` equivale a **falso** y cualquier otro número (positivo o negativo) equivale a **verdadero**.

También es posible usar **múltiples condiciones encadenadas**:

```
int a = 1;
if (a == 2) {
  /* hacer algo */
} else if (a == 1) {
  /* hacer otra cosa */
} else {
  /* hacer una tercera cosa */
}
```

switch

Cuando se necesita evaluar una variable contra múltiples valores posibles, usar muchos `if / else if` puede ser engorroso. En esos casos, la sentencia `switch` es más conveniente.

```
int a = 1;
switch (a) {
                        case 0:
                          /* hace algo si a = 0 */
                          break;
                        case 1:
                          /* hace otra cosa si a = 1 */
                          break;
                        case 2:
                          /* hace otra cosa si a = 2 */
                          break;
                        default:
                          /* aquí se manejan todos los
     otros casos */
                          break;
}
```

Uso de break: Cada case suele terminar con break para **evitar que la ejecución continúe** en el siguiente caso. Si se omite break, ocurre el efecto de **"cascada"**, donde los bloques siguientes también se ejecutan. Esto puede usarse de manera intencional en

algunos diseños.

Caso por defecto: Es posible definir un **bloque por defecto** con default para manejar cualquier valor que no coincida con los casos anteriores

Bucles en C

C nos ofrece **tres tipos principales de bucles**:

1. `for` (repetitiva exacta)
2. `while` (repetitiva 0 a n)
3. `do while` (repetitiva 1 a n)

Ciclo for

El ciclo `for` es probablemente el más común y permite definir toda la configuración del bucle en una sola línea:

```
for (int i = 0; i <= 10; i++) {
  printf("%d ", i);
}
```

La cabecera del bucle contiene:

- **Inicialización** → `int i = 0`
- **Condición** → `i <= 10`
- **Incremento** → `i++`

En el ejemplo se imprime la variable i del 0 al 10.

Ciclo while

El ciclo `while` ejecuta un bloque mientras se cumpla una condición:

```
int i = 0;
while (i < 10) {
                        /* hacer algo */
                        i++;
}
```

Nota: Si no se modifica la condición dentro del bucle, puede crearse un **bucle infinito** que bloqueará el programa. En este caso si no se incrementa i mediante **i++** valdría lo mismo **i** en cada iteración, creando un bucle infinito.

Ciclo do while

La variante `do while` es similar al `while`, pero **garantiza al menos una ejecución del bloque**, incluso si la condición es falsa desde el inicio:

```
int i = 0;
do {
                        /* haz algo */
                        i++;
} while (i < 10);
```

Saliendo de un bucle con `break`

En cualquier bucle (`for`, `while` o `do while`), la palabra clave `break` permite salir inmediatamente, sin esperar a que la condición deje de cumplirse.

```
for (int i = 0; i <= 10; i++) {
                        if (i == 4 && algunaVariable ==
     10) {
                          break;
                        }

                      // otras instrucciones
}
```

Esto es especialmente útil en bucles que podrían ser infinitos.

```
int i = 0;
while (1) {   // bucle infinito
                        /* hacer algo */
                        i++;
                        if (i == 10)
                                  break;    // condición
     de salida
}
```

Punteros

En C, un puntero es una variable especial que no guarda directamente un valor, sino la dirección de memoria donde se encuentra ese valor.

Esto permite trabajar de forma más cercana al hardware, manipular estructuras dinámicas y optimizar el uso de memoria.

Un puntero debe declararse siempre indicando a qué tipo de dato apunta. Esto asegura que el compilador sepa cuántos bytes leer o escribir al acceder al valor.

Declaración básica:

```
int *p;     // puntero a entero
float *q;   // puntero a flotante
char *c;    // puntero a caracter
```

Operadores asociados a punteros

1. **Operador de referencia (&):**

   Devuelve la dirección de memoria de una variable.

```
int x = 10;
int *p = &x;   // p guarda la dirección de x
```

2. **Operador de desreferencia (*):**

   Permite acceder al contenido de la dirección almacenada en el puntero.

```
printf("%d", *p);  // imprime 10, valor almacenado en x
```

En resumen:

- **&x** significa "l**a dirección de x**".
- ***p** significa "**el valor apuntado por p**".

Ejemplo completo

```
#include <stdio.h>

int main() {
    int x = 42;
    int *p = &x;

    printf("Valor de x: %d\n", x);
    printf("Direccion de x: %p\n", &x);
    printf("Valor guardado en p: %p\n", p);
    printf("Valor apuntado por p: %d\n", *p);

    *p = 100;  // modificamos x a través del puntero
    printf("Nuevo valor de x: %d\n", x);

    return 0;
}
```

En este ejemplo, se ve que los punteros no solo sirven para leer, sino también para modificar indirectamente el valor de una variable.

Especificador de formato %p: Cuando queremos imprimir direcciones de memoria con printf(), debemos usar el especificador **%p**. Este formato está diseñado específicamente para mostrar punteros y direcciones en hexadecimal.

Arreglos

Un arreglo es una colección de variables del mismo tipo, almacenadas en posiciones contiguas de memoria. Permiten organizar y manejar conjuntos de datos de manera eficiente.

Arreglos unidimensionales (Vectores)

Son la forma más simple de arreglo. Se indexan desde 0 hasta n-1.

```c
#include <stdio.h>

int main() {

    int v[5] = {10, 20, 30, 40, 50};

    for(int i=0; i<5; i++){

        printf("%d\n", v[i]);  // imprime cada elemento de v

    }

    return 0;
}
```

Arreglos bidimensionales (Matrices)

Se utilizan para representar tablas de datos.

```c
int m[2][3] = {
  {1, 2, 3},
  {4, 5, 6}
};
printf("%d\n", m[0][2]); // imprime 3
printf("%d\n", m[1][0]); // imprime 4
```

Arreglos multidimensionales

Más de dos dimensiones, poco comunes salvo en casos como procesamiento de imágenes o simulaciones.

```c
int t[2][2][2] = {
  { {1,2}, {3,4} },
  { {5,6}, {7,8} }
};
printf("%d\n", t[1][0][1]); // imprime 6
```

El nombre de un vector en C

En C, cuando usamos el nombre de un vector (por ejemplo v), este se interpreta como la dirección de memoria donde comienza el arreglo, es decir, la dirección de su primer elemento

```c
#include <stdio.h>

int main() {
    int v[5] = {1,2,3,4,5};

    printf("%p\n", v);   // dirección del primer elemento
    printf("%d\n", *v);  // valor del primer elemento (desreferencia)

    return 0;
}
```

El nombre de un vector entonces es equivalente a un puntero al primer elemento.

- v → dirección de v[0]
- *v → contenido de v[0]

Recorrido de un vector

El bucle **for** es ideal para recorrer arreglos, ya que conocemos exactamente la cantidad de elementos. En el siguiente ejemplo se muestra el recorrido de un vector.

```c
#include <stdio.h>

int main() {
    int v[4]= { 10, 20, 30, 40 };

    for (int i = 0; i < 4; i++) {        // recorre cada elemento

            printf("v[%d] = %d\n", i, v[i]);

    }
    return 0;
}
```

➢ ⚠️ **Importante:** En C, los arreglos no verifican automáticamente sus límites. Si se accede a un índice fuera del rango válido (0 a n-1), el programa no mostrará error, pero el resultado será comportamiento indefinido: valores incorrectos, sobrescritura de memoria u fallos inesperados.

Por eso, es responsabilidad del programador asegurarse de usar siempre índices dentro del rango. Una práctica recomendada es controlar los límites en bucles y definir el tamaño de los arreglos con constantes o macros para evitar errores.C no verifica los límites de los arreglos en tiempo de ejecución, acceder fuera de rango es comportamiento indefinido.

Recorrido de una matriz

En el caso de una matriz, se necesitan dos bucles for anidados:

- El índice externo recorre las filas.
- El índice interno recorre las columnas.

```c
#include <stdio.h>

int main() {
    int m[2][3] = { {1, 2, 3}, {4, 5, 6} };

    for (int fil = 0; fil < 2; fil++) {        // recorre filas
        for (int col = 0; col < 3; col++) {    // recorre columnas
            printf("m[%d][%d] = %d\n", fil, col, m[fil][col]);
        }
    }

    return 0;
}
```

Búsqueda lineal en un vector

La búsqueda lineal consiste en revisar uno a uno los elementos de un vector hasta encontrar el valor buscado.

```c
#include <stdio.h>

int main() {

     int v[6] = {8, 3, 7, 1, 9, 4};
     int buscado = 7;    //nro a buscar
     int encontrado = -1;

     for (int i = 0; i < 6; i++) {
          if (v[i] == buscado) {
               encontrado = i;
               break;   // dejamos de buscar
          }
     }

     if (encontrado != -1)   //encontrado = -1 indica que no fue hallado
          printf("Valor encontrado en la posicion %d\n", encontrado);
     else
          printf("Valor no encontrado\n");

     return 0;
}
```

Ordenamiento (Bubble Sort)

El Bubble Sort (ordenamiento burbuja) es uno de los algoritmos más sencillos para ordenar. Funciona comparando elementos adyacentes y, si están en el orden incorrecto, los intercambia. Con cada pasada, el mayor “burbujea” hasta el final.

```c
#include <stdio.h>

int main() {
    // Vector de ejemplo con elementos desordenados
    int v[5] = {5, 1, 4, 2, 8};
    int n = 5;   // cantidad de elementos

    // Recorremos varias "pasadas" sobre el vector
    for (int i = 0; i < n-1; i++) {
        // En cada pasada, el mayor de los elementos restantes
        // "burbujea" hasta el final del vector.
        for (int j = 0; j < n-i-1; j++) {
            // Comparar elementos adyacentes
            if (v[j] > v[j+1]) {
                // Si están en el orden incorrecto, intercambiarlos
                int temp = v[j];
                v[j] = v[j+1];
                v[j+1] = temp;
            }
        }
    }

    // Mostrar el vector ya ordenado
    printf("Vector ordenado: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", v[i]);
    }
    printf("\n");

    return 0;
}
```

Explicación paso a paso:

1. **Inicialización**: Se define un vector con valores desordenados {5, 1, 4, 2, 8}.

2. **Bucle externo (i)**: Controla la cantidad de pasadas. Después de cada pasada, el último elemento queda en su posición definitiva.

3. **Bucle interno (j)**:Recorre los elementos del vector comparando cada par de adyacentes.

4. **Comparación e intercambio**: Si el elemento actual es mayor que el siguiente, se intercambian usando una variable auxiliar temp.

5. **Resultado final**: El vector queda ordenado de menor a mayor.

Funciones y Modularización en C

¿Por qué modularizar?

Un programa lineal puede crecer hasta volverse inmanejable. La solución es dividirlo en funciones que resuelven subtareas específicas. Esto permite:

- Claridad y orden.
- Reutilización de código.
- Escalabilidad y mantenimiento.

En C, todo son funciones. Algunas devuelven un valor (funciones propiamente dichas) y otras solo realizan una acción (procedimientos).

Funciones

Una función devuelve un valor con `return`.

```
tipo nombre_funcion(tipo1 param1, tipo2 param2);

int sumar(int a, int b) {
    return a + b;
}
```

Procedimientos

En C también son funciones, pero declaradas con `void`, ya que no devuelven valor.

```
void imprimirSuma(int a, int b) {
    printf("La suma es: %d\n", a + b);
}
```

Tipo de dato `void`

- `void` significa “sin valor”.
- Usos:
    - Funciones que no retornan valor.
    - Funciones sin parámetros (`int main(void)`).
    - Punteros genéricos (`void *`).

Ejemplo de puntero genérico:

```
void *ptr;
int x = 10;
ptr = &x;   // ptr puede apuntar a cualquier tipo
```

Variables locales y globales

- **Locales**: declaradas dentro de una función. Solo existen durante su ejecución.
- **Globales**: declaradas fuera de `main()`. Visibles en todo el programa.

⚠️ Usar variables globales con cuidado: pueden generar errores difíciles de rastrear.

Pasaje de parámetros

- **Por valor**: se envía una copia de la variable. No modifica el original.
- **Por referencia**: se envía la dirección de memoria (puntero). La función puede modificar el original.

  Procedimiento duplicar, pasaje por referencia

```
void duplicar(int *x) {
    *x = (*x) * 2;
}
```

  Llamada a duplicar

```
void duplicar(int *x) {
    *x = (*x) * 2;
}
```

Funciones recursivas

Una función puede llamarse a sí misma.

Ejemplo clásico: **factorial**

```
int factorial(int n) {
    if (n == 0) return 1;   // caso base
    return n * factorial(n - 1);
}
```

Buenas prácticas

- Dividir el problema en funciones pequeñas.
- Usar nombres descriptivos.
- Evitar variables globales.
- Usar void para claridad en funciones sin retorno.
- Controlar siempre las condiciones de corte en recursividad.